



الوب وبروتوكول HTTP

رقم الصفحة	العنوان
2	1. لمحة عن البروتوكول HTTP
3	1.1. الزبون (المتصفح)
4	2.1. المخدم
4	3.1. عنوان مورد نظامي (URL) Uniform Resource Locator
5	4.1. وثائق الوب
7	2. بروتوكول HTTP
7	1.2. مداولات HTTP
13	2.2. استخدام Cookies
14	3.2. المخدم الوكيل Proxy server وتخبيئة الوب Web caching
16	3. عملي
16	1.3. رسائل Get/response
17	2.3. استخدام HTTP conditional Get/response
18	3.3. عرض ملف كبير
18	4.3. استيقان HTTP
21	5.3. إدارة الكوكيز

كان استخدام الإنترنت، حتى بداية التسعينيات، محصوراً بالباحثين والأكاديميين وطلاب الجامعات بغية النفاذ عن بعد إلى حواسيب بعيدة أو لنقل الملفات أو لتبادل البريد الإلكتروني أو للمناقشة ضمن ما كان يعرف بخدمة الأخبار. جذب دخول الوب إلى الإنترنت انتباه عموم الناس وساهم في تغيير النظرة إلى الإنترنت وطرق التعامل معها الأمر الذي ساهم في تحويل الإنترنت من واحدة من الشبكات الموجودة إلى الشبكة الوحيدة المهيمنة. ما يميز الوب هو كون الخدمة حسب الطلب on-demand. يستقبل المستخدمون ما يريدون وفي الوقت الذي يريدون بعكس الخدمات التي تقدمها وسائل بث الراديو والتلفاز التي تجبر المتلقين على المتابعة عندما يريد المصدر بدء البث. أضف إلى ذلك أن أي شخص بات قادراً أن يصبح ناشراً بدون كلفة كبيرة. كما ساعدت محركات البحث والنصوص المترابطة في تسهيل عملية الملاحاة ضمن الإنترنت. تجدر الإشارة هنا إلى انتشار بعض التطبيقات مثل YouTube, Gmail, and Facebook والتي جذبت أعداداً كبيرة من المستخدمين بعد عام 2000.

1. لمحة عن البروتوكول HTTP

يعتبر بروتوكول (HTTP) HyperText Transfer Protocol، وهو بروتوكول الوب الذي يعمل على مستوى طبقة التطبيقات، قلب الوب. يعرف هذا البروتوكول ضمن [RFC 1945] و [RFC 2616]. جرى تحقيق HTTP باستخدام برنامجين: برنامج زبون وبرنامج مخدم. يتخاطب هذين البرنامجين مع بعضهما البعض باستخدام رسائل HTTP. يعرف HTTP بنية الرسائل وطريقة تبادلها. لنذكر بعض المصطلحات المستخدمة ضمن الوب:

تتألف **صفحة الوب Web page** (أو الوثيقة) من مجموعة من الأغراض. الغرض هو ملف كملف HTML أو صورة JPEG أو مقطع فيديو أو بريمج Java applet والذي يمكن عنوانته باستخدام Uniform Resource Locator (URL) وحيد.

تتألف معظم صفحات الوب من ملف HTML أساسي base HTML file ومجموعة من الأغراض المرتبطة بها referenced objects. فمثلاً، إذا كانت صفحة الوب تحوي ملف HTML وخمس صور JPEG فهذا يعني أن صفحة الوب تحوي ستة أغراض: الملف الأساسي مضاف إليه خمس صور. يؤثر الملف الأساسي إلى الصور الخمسة باستخدام URL لكل غرض.

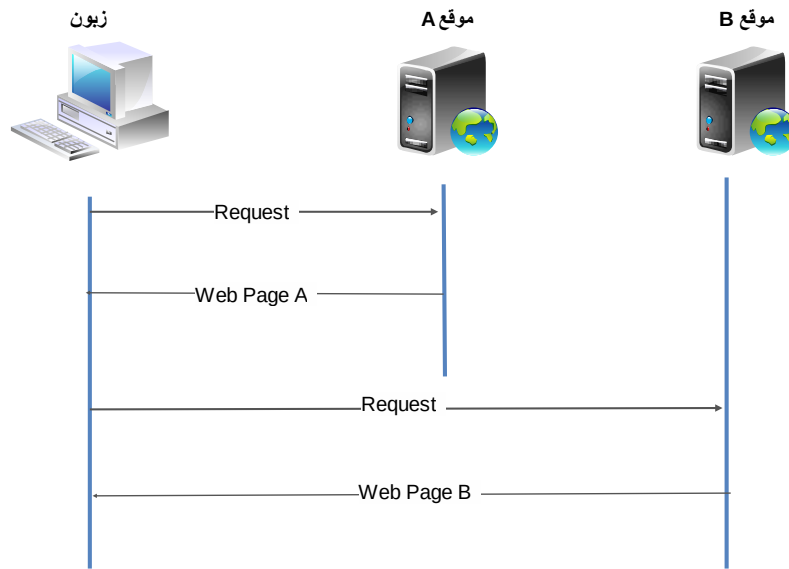
بما أن تحقيق زبون الوب يكون عادةً ضمن متصفح الإنترنت مثل Firefox فيمكننا استخدام المصطلحين معاً للدلالة على زبون الوب.

من الأمثلة عن مخدمات الوب المشهورة هناك Apache و Microsoft Internet Information Server (IIS).

عندما يضغط المستخدم على رابط ما أو يدخل عنوان URL ضمن متصفح الإنترنت فإن المتصفح يرسل رسالة طلب HTTP إلى المخدم. يستقبل المخدم رسالة الطلب ويعيد رسالة الجواب Response message والتي تحوي على الغرض المطلوب.

يعتمد بروتوكول HTTP على طبقة TCP لنقل المعلومات المتبادلة مع الزبون. يبدأ زبون HTTP أولاً بفتح اتصال TCP مع المخدم. بعد ذلك، تمر جميع الرسائل المتبادلة بين الطرفين عبر هذا الاتصال. من المهم الانتباه إلى أن مخدم الويب لا يحتفظ بأي معلومات عن حالة الزبون بعد تقديمه. فإذا طلب الزبون الغرض نفسه مرتين متتاليتين، فالمخدم غير قادر على معرفة ذلك وسيعيد له الغرض المطلوب كل مرة. لذلك يعرف بروتوكول HTTP بأنه عديم الحالة Stateless protocol. يستخدم الويب نموذج زبون-مخدم كما أن المخدم يكون دائماً قيد العمل ومزود بعنوان IP ساكن وهو قادر على تقديم عدة مستخدمين في الوقت نفسه.

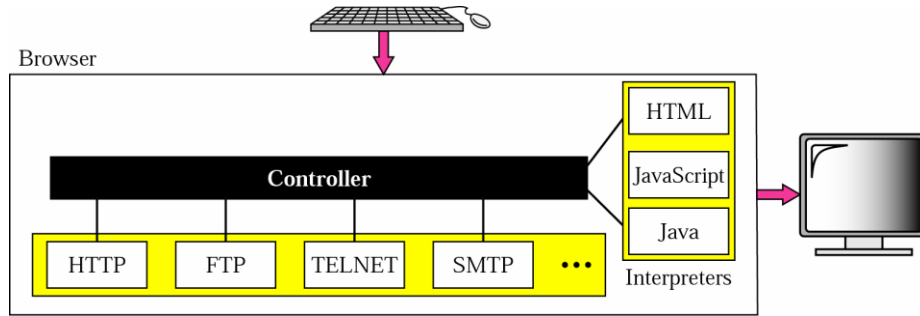
إن شبكة الويب هي خدمة موزعة من نوع مخدم-زبون، حيث يستطيع الزبون، عن طريق المتصفح، النفاذ إلى خدمة باستخدام المخدم. إلا أن الخدمة المزودة تكون موزعة على عدة مواقع كما هو موضح في الشكل التالي.



الشكل 1- خدمة الويب الموزعة

1.1. الزبون (المتصفح)

يعتبر المتصفح مسؤولاً عن تفسير وعرض صفحات الويب. يتألف كل متصفح من ثلاثة أجزاء: متحكم Controller وبروتوكول الزبون ومفسر interpreter. يستقبل المتحكم الدخول من لوحة المفاتيح أو الفأرة ويستخدم برامج الزبون للنفاذ إلى الوثيقة. يستخدم بعد ذلك المتحكم أحد المفسرات المتوفرة لعرض الوثيقة على الشاشة. يمكن أن يكون بروتوكول الزبون FTP أو Telnet أو HTTP. أما المفسر، فيمكن أن يكون HTML أو جافا أو Javascript حسب نوع الوثيقة. يبين الشكل التالي تصميمًا توضيحيًا لبنية المتصفح.



الشكل 2- بنية المتصفح

2.1. المخدم

تُخزن صفحة الويب على المخدم. في كل مرة يرسل الزبون لطلب ما فإن المخدم يقوم بإرسال الصفحة المقابلة. تخزن المخدمات عادةً الصفحات المطلوبة ضمن ذاكرة خابية لتحسين الأداء. يمكن أيضاً زيادة فعالية المخدم باستخدام تقانات مثل multithreading أو multitasking حيث يصبح المخدم قادراً على معالجة عدة طلبات في الوقت نفسه.

3.1. عنوان مورد نظامي (URL) Uniform Resource Locator

يحتاج الزبون إلى عنوان صفحة وب حتى يستطيع النفاذ إليها. يستخدم بروتوكول HTTP مفهوم العنوان Locator لتسهيل عملية النفاذ إلى الوثائق الموزعة على العالم. يعتبر عنوان URL معياراً لتوصيف أي نوع من الوثائق الموجودة على الإنترنت. يعرف عنوان URL أربعة أشياء: البروتوكول والحاسب المضيف والبوابة والمسار. يأخذ URL الصيغة التالية:

Protocol://Host:Port/Path

البروتوكول هو برنامج مخدم-زبون المستخدم لاستحضار الوثيقة. يوجد عدة بروتوكولات قادرة على استحضار الوثائق مثل Gopher, FTP, HTTP, News, Telnet. لكن HTTP هو الأكثر شهرةً.

المضيف host هو الحاسب الموجودة عليه الوثيقة أو المعلومة المطلوبة. يمكننا هنا استخدام اسم مستعار alias للمضيف يبدأ عادةً بـ www لكن ذلك غير إجباري.

يمكن أن تحوي URL على رقم بوابة المخدم.

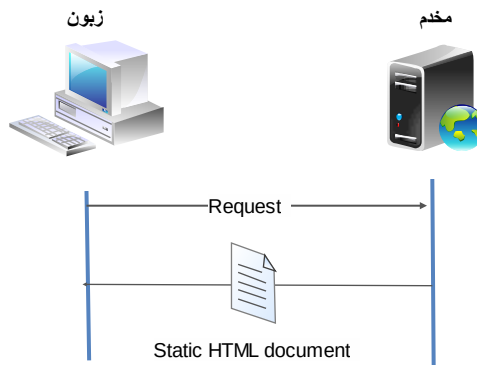
المسار path هو اسم المسار للوصول إلى الملف الذي يحوي الوثيقة المطلوبة. لاحظ أنه يمكن للمسار أن يحوي على slashes التي تفصل، حسب نظام التشغيل UNIX، الدليل عن الدلائل الفرعية أو الملفات.

4.1. وثائق الوب

يمكن تصنيف الوثائق ضمن شبكة الوب إلى ثلاثة أصناف: الساكنة static والديناميكية dynamic والنشطة active. يعتمد التصنيف على زمن تحديد محتوى الوثيقة.

الوثائق الساكنة Static documents

الوثائق الساكنة هي وثائق ثابتة المحتوى، يجري توليدها وتخزينها على المخدم. يمكن للزبون الحصول على نسخة من الوثيقة فقط. أي أنه يجري تحديد محتوى الوثيقة لحظة توليدها وليس لحظة استخدامها. طبعاً، يستطيع المخدم فقط تغيير محتوى الوثيقة أما الزبون فيستطيع، باستخدام المتصفح، عرض محتويات الوثيقة (انظر الشكل التالي).



الشكل 3- الوثائق الساكنة

لغة تأشير نص ترابطي (Hypertext Markup Language) (HTML)

HTML هي لغة تسمح بإنشاء صفحات الوب. يفيد التأشير في صياغة صفحة الوب ليستطيع المتصفح تفسيرها ومن ثم عرضها.

تسمح لغات التأشير مثل HTML بتضمين تعليمات صياغة ضمن النص نفسه. بهذه الطريقة، يستطيع أي متصفح قراءة التعليمات وصياغة النص حسب محطة العمل.

تسمح HTML باستخدام محارف ASCII لكتابة النص نفسه ولتعليمات الصياغة.

تتألف صفحة الوب من قسمين: الرأس والمحتوى. يحوي الرأس عنوان الصفحة وبعض المتحولات التي يمكن للمتصفح استخدامها. يوضع المحتوى الفعلي للصفحة ضمن المتن الذي يحوي النص والأوامر tags. تُعرّف الأوامر طريقة إظهار النص. تتكون كل أمانة من اسم ومجموعة اختيارية من الخاصيات، توضع جميعاً ضمن إشارتي أصغر وأكبر (< and >).

عندما توجد الخاصية attribute فإنه يليها إشارة المساواة (=) وقيمة الخاصية. يمكن استخدام بعض الأوامر بشكل وحيد بينما يُستخدم البعض الآخر كزوج (أمانة البداية وأمانة النهاية). يمكن لأمانة البداية أن تمتلك خاصيات أو قيم وتبدأ باسم الأمانة. لا يمكن لأمانة النهاية أن تمتلك خاصيات أو قيم لكن يجب وضع slash (/) قبل اسم الأمانة. يبين الشكل التالي صيغة الأمانة.

< TagName Attribute = Value Attribute = Value ... >

أ – أمانة البداية Beginning tag

< /TagName >

ب – أمانة النهاية Ending tag

الشكل 4 – أمانة البداية والنهاية

من أشهر الأمانة المستخدمة لصياغة النصوص:

- و لإظهار النص بالبنط العريض.
- <I> و </I> لإظهار النص بالبنط المائل.
- <U> و </U> لإظهار النص تحته خط.

الوثائق الديناميكية

يجري توليد الوثيقة الديناميكية من قبل مخدم الويب عندما يطلب المتصفح الصفحة. عندما يصل الطلب، ينفذ المخدم برنامج تطبيقي ما أو سكريبت يسمح بتوليد الصفحة الديناميكية. يعيد المخدم خرج البرنامج أو خرج السكريبت كجواب على طلب المتصفح. بما أنه يجري توليد صفحة وب جديدة لكل طلب، فيمكن أن تكون الصفحات مختلفة من طلب إلى آخر. لعل أفضل مثال على ذلك، متصفحات تطلب التاريخ والوقت من مخدم.

واجهة العبارة المشتركة (CGI) Common Gateway Interface

إن واجهة العبارة المشتركة هي تقانة تسمح بتوليد ومعالجة الوثائق الديناميكية. CGI هي مجموعة من المعايير التي تُعرّف كيف يجري كتابة وثيقة ديناميكية، وكيف يجري إدخال المعطيات إلى البرنامج، وكيف يجري استخدام نتائج الخرج.

تعرف CGI مجموعة من القواعد والمصطلحات التي يجب على المبرمج احترامها.

الوثائق النشطة Active documents

نحتاج في العديد من التطبيقات إلى تنفيذ برنامج أو سكريبت على موقع الزبون. هذا ما يدعى بالوثيقة النشطة. لنأخذ على سبيل المثال تنفيذ برنامج لتوليد بيانات حية animated graphics على الشاشة أو برنامج آخر يتفاعل مع المستثمر. عندما يطلب المتصفح وثيقة نشطة فإن المخدم يرسل نسخة عن الوثيقة أو السكريبت. يجري الآن تنفيذ الوثيقة على حاسب الزبون (المتصفح).

2. بروتوكول HTTP

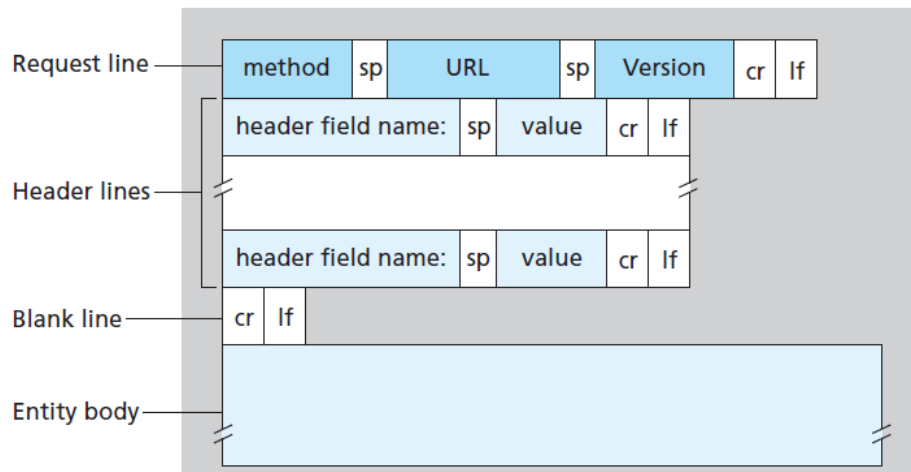
يُستخدم بروتوكول HTTP بشكل أساسي للنفاز إلى المعطيات الموجودة على الشبكة العنكبوتية. يعمل بروتوكول HTTP كمزيج من FTP و SMTP. يشبه FTP لأنه ينقل الملفات ويستفيد من خدمات TCP. لكنه أبسط بكثير من FTP لأنه يستخدم ارتباط TCP واحد لنقل المعطيات بين المخدم والزيون. يشبه أيضاً بروتوكول SMTP لأن المعطيات المنقولة بين المخدم والزيون تشبه رسائل SMTP. لكن بعكس SMTP فإن الرسائل غير معدة للقراءة من قبل الإنسان؛ لأنها تُقرأ وتُفسر من قبل مخدم أو زيون HTTP إضافةً إلى إرسال الرسائل مباشرةً دون تخزين وإعادة توجيه كما هو عليه الحال ضمن SMTP. تُضمّن الطلبات من الزيون إلى المخدم ضمن رسالة طلب Request message بينما يُضمّن محتوى الملف المطلوب ضمن رسالة الجواب Response message. يستخدم HTTP خدمات TCP على البوابة المعروفة 80.

1.2. مداولات HTTP

مع أن HTTP يستخدم خدمات TCP إلا أنه بروتوكول عديم الحالة Stateless. يهيئ الزيون المداولة عن طريق إرسال رسالة الطلب ويرد المخدم عن طريق إرسال الجواب.

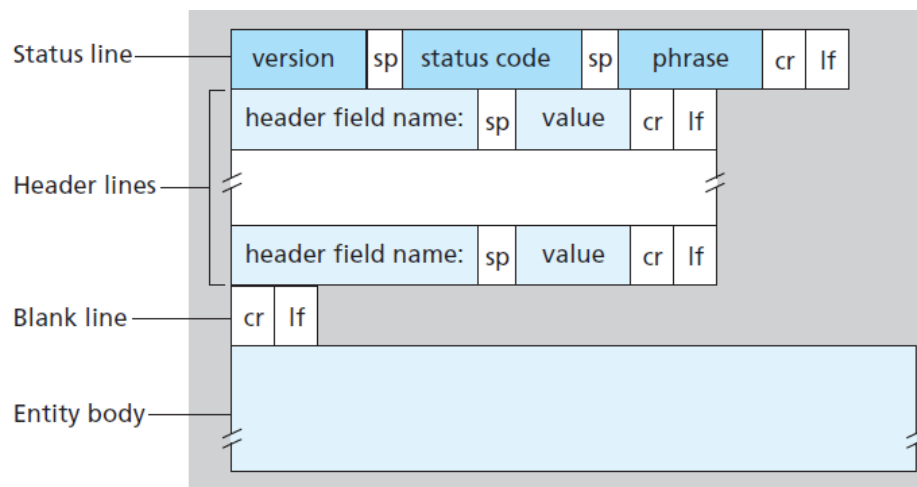
الرسالة

يبين الشكل التالي صيغة الطلب.



الشكل 5- رسالة الطلب

ويبين الشكل التالي رسالة الجواب:

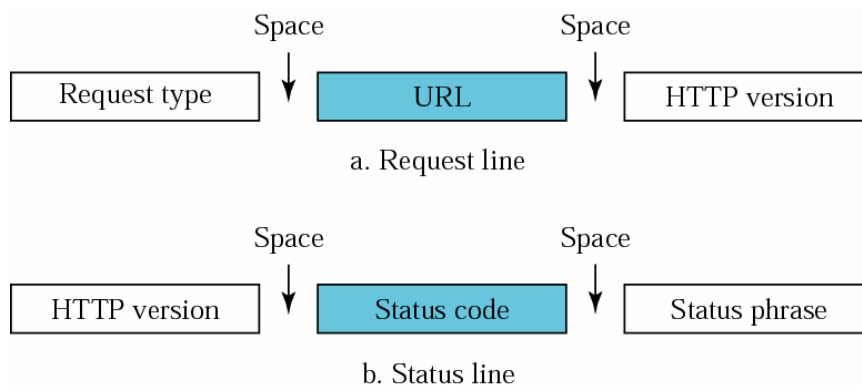


الشكل 6- رسالة الجواب

تتكون رسالة الطلب من سطر الطلب وترويسة وفي بعض الأحيان، متن. أما رسالة الجواب فتتكون من سطر الحالة وترويسة وفي بعض الأحيان، متن.

أسطر الطلب والحالة Request and Status lines

يبين الشكل التالي صيغة هذين السطرين.



الشكل 7- أسطر الطلب والجواب

- **نوع الطلب request type.** جرى تعريف عدة أنواع خدمة ضمن HTTP 1.1. يُصنف نوع الخدمة حسب طرائق methods كما هو مبين في الشكل التالي.

Method	Action
GET	طلب وثيقة من المخدم
HEAD	طلب معلومات عن وثيقة لكن ليس الوثيقة نفسها
POST	إرسال الزبون لبعض المعلومات على المخدم
PUT	إرسال المخدم لوثيقة إلى الزبون
TRACE	إعادة echo الطلب الداخل
CONNECT	محجوزة
OPTION	الاستفسار عن الاختيارات المتاحة

الشكل 8- الطرائق methods

- **URL.** أصبح معروفاً الآن.
- **Version.** أحدث إصدار للبروتوكول هو HTTP1.1.
- **رمز الحالة Status code.** يستخدم هذا الحقل ضمن رسالة الجواب. يتألف رمز الحالة من 3 أرقام. تكون جميع الرموز ضمن مجال 100 هي رموز إعلامية informational فقط؛ تشير الرموز ضمن مجال 200 إلى نجاح الطلب؛ تعيد الرموز ضمن مجال 300 توجيه الزبون إلى عنوان URL آخر؛ تشير الرموز ضمن مجال 400 إلى خطأ في طرف الزبون وضمن المجال 500 إلى خطأ في طرف المخدم. يبين الجدول التالي الرموز الأكثر شيوعاً.

Code	Phrase	الشرح
Informational		
100	Continue	جرى استقبال القسم الأول من الطلب ويمكن للزبون المتابعة في طلبه
101	Switching	يستجيب المخدم لطلب الزبون لتبديل البروتوكولات المعرفة داخل ترويسة التحديث
Success		
200	OK	نجاح الطلب
201	Created	لقد جرى توليد URL جديدة
202	Accepted	تم قبول الطلب، لكن لا يجر تنفيذها حالياً

204	No content	لا يوجد محتوى ضمن المتن
Redirection		
301	Moved permanently	لم يعد المخدم يستخدم الـ URL المطلوبة
302	Moved temporarily	تم نقل الـ URL المطلوبة مؤقتاً
304	Not modified	لم يجر تعديل الوثيقة
Client error		
400	Bad request	يوجد خطأ في التركيب النحوي للطلب
401	Unauthorized	ينقص الطلب أذن الدخول
403	Forbidden	الخدمة ممنوعة
404	Not found	الوثيقة غير موجودة
405	Method not allowed	الطريقة غير مدعومة ضمن هذه الـ URL
406	Not accepted	الصيغة المطلوبة غير مقبولة
Server error		
500	Internal server error	يوجد خطأ على موقع المخدم أو كونه معطل
501	Not implemented	لا يمكن تحقيق العملية المطلوبة
503	Service unavailable	الخدمة غير متاحة مؤقتاً، لكن يمكن إعادة الطلب لاحقاً

الشكل 9- رموز الحالة

الترويسة Header

تستخدم الترويسة لتبادل معلومات إضافية بين المخدم والزيون، كأن يطلب الزيون إرسال الملف بصيغة خاصة أو أن يرسل المخدم معلومات إضافية عن الملف. تتكون الترويسة من سطر أو أكثر. يحوي كل سطر اسم الترويسة وعلامة الترقيم ":" وفراغ وقيمة للترويسة. أي Header name: Header value.

ينتمي سطر الترويسة إلى أحد الأصناف التالية: **ترويسة عامة، ترويسة طلب، ترويسة جواب، ترويسة كيان**. يمكن أن تحوي رسالة الطلب Request message إما الترويسة العامة أو الطلب أو الكيان. أما رسالة الجواب فيمكن أن تحوي الترويسة العامة أو الجواب أو الكيان.

- **الترويسة العامة General header**. تعطي معلومات عامة عن الرسالة. يبين الشكل التالي بعض الترويسات العامة.

Header	الوصف
Cache-control	توصيف معلومات عن caching
Connection	إظهار إذا كان يجب إغلاق الارتباط أو لا
Date	إظهار التاريخ الحالي
MIME-version	إظهار إصدار MIME
Upgrade	توصيف بروتوكول الاتصال المفضل

الشكل 10- الترويسات العامة

- **ترويسة الطلب Request header.** تحدد إعدادات الزبون وصيغة الوثائق التي يفضلها الزبون. يبين الشكل التالي بعض ترويسات الطلب.

Header	الوصف
Accept	إظهار صيغة الوسائط التي يقبلها الزبون
Accept-charset	إظهار طقم المحارف التي يستطيع الزبون معالجتها
Accept-encoding	إظهار الترميز الذي يستطيع الزبون معالجته
Accept-language	إظهار اللغة التي يستطيع الزبون قبولها
Authorization	إظهار السماحيات التي يمتلكها الزبون
From	إظهار العنوان البريدي الإلكتروني للمستثمر
Host	إظهار اسم المضيف ورقم البوابة للزبون
If-modified-since	أرسل الوثيقة إذا كانت أحدث من تاريخ محدد
If-match	أرسل الوثيقة إذا كانت تطابق أمانة محددة
If-not-match	أرسل الوثيقة إذا كانت لا تطابق أمانة محددة
If-range	أرسل الجزء الناقص من الوثيقة
If-unmodified-since	أرسل الوثيقة إذا لم تتغير منذ التاريخ المحدد
Referrer	توصيف URL للوثيقة المرتبطة
User-agent	تعريف برنامج الزبون

الشكل 11- ترويسات الطلب

- **ترويسة الجواب Response header.** تحدد إعدادات المخدم ومعلومات خاصة عن الطلب. يبين الشكل التالي بعض ترويسات الجواب.

Header	الوصف
Accept-range	إظهار إذا كان المخدم يقبل المجال الذي حدده الزبون
Age	إظهار عمر الوثيقة
Public	إظهار لائحة بالطرائق المدعومة
Retry-after	تحديد الوقت الذي بعده يصبح المخدم متاح
Server	إظهار اسم المخدم ورقم الإصدار

الشكل 10- ترويسات الجواب

- **ترويسة الكيان Entity header.** تعطي ترويسة الكيات معلومات عن متن الوثيقة. توجد هذه الترويسة غالباً ضمن رسائل الجواب غير أن بعض رسائل الطلب التي تحوي متن مثل PUSH و PUT تستخدم هذه الترويسة. يبين الشكل التالي بعض ترويسات الكيان.

Header	الوصف
Allow	إدراج الطرائق المقبولة التي يمكن استخدامها مع URL
Content-encoding	تحديد طريقة الترميز
Content-language	تحديد اللغة
Content-length	تحديد طول الوثيقة
Content-range	تحديد مجال الوثيقة
Content-type	تحديد نوع الوسائط
Etag	إعطاء أمانة للكيان Entity tag
Expires	تحديد تاريخ ووقت انتهاء صلاحية المحتوى
Last-modified	تحديد تاريخ ووقت التعديل الأخير
Location	تحديد موضع الوثيقة المولدة أو المنقولة

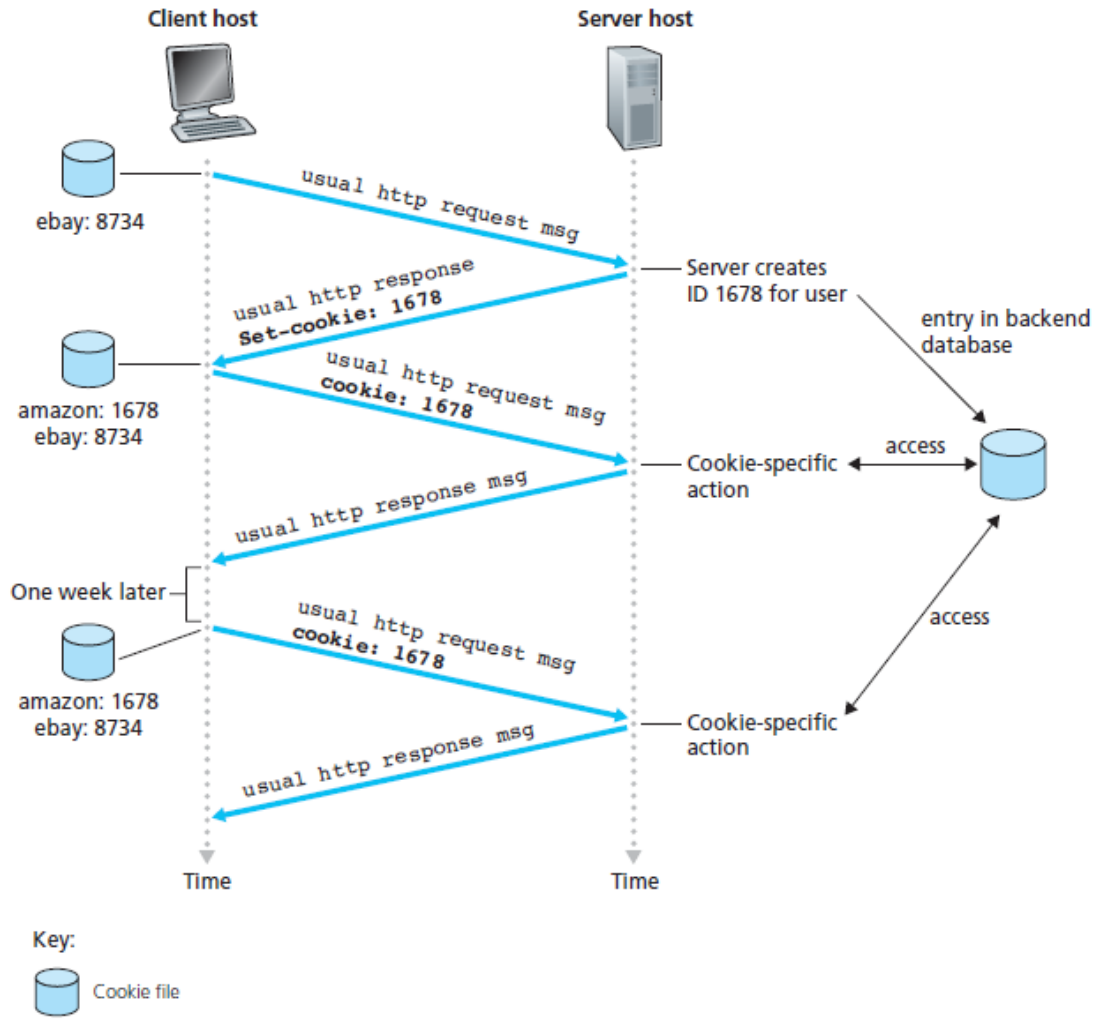
الشكل 11- ترويسات الكيان

- **المتن Body.** يمكن أن يكون المتن موجوداً ضمن رسالة الطلب أو رسالة الجواب. يحوي عادةً المعلومات المطلوب إرسالها أو توصيلها.

2.2. استخدام Cookies

لقد قلنا سابقاً أن بروتوكول HTTP هو عديم الحالة، طبعاً هذا يفيد في تبسيط العمليات التي يقوم بها الأمر الذي يحسن أداء المخدم مما يسمح له بمعالجة آلاف الطلبات في الوقت نفسه. لكن مخدم الويب، في بعض الحالات، يحتاج إلى التعرف على المستثمرين ومتابعتهم بغية توجيه المحتوى حسب المستثمر مثلاً. لذلك تقوم مخدمات HTTP باستخدام cookies. تسمح الكوكيز، حسب المعيار [RFC 6265]، للمواقع بمتابعة المستثمرين. كما أن معظم مواقع الإنترنت التجارية الحالية تستخدم الكوكيز. تتألف تقنية الكوكيز من أربعة مكونات:

- وضع الكوكي ضمن سطر ترويسة رسالة الجواب
 - وضع ترويسة الكوكي ضمن رسالة الطلب
 - يحتفظ متصفح الإنترنت بالكوكي ويقوم بإدارتها
 - يقوم مخدم الويب بإضافة مدخل ضمن قاعدة المعطيات خاصة بهذه الكوكي
- يبين الشكل التالي تسلسل عمليات إنشاء الكوكيز ومتابعتها ضمن المتصفح أو المخدم.



الشكل 14- متابعة حالة المستثمر باستخدام الكوكيز

نلاحظ من الشكل السابق أن المستثمر كان يملك كوكي سابقة برقم 8734 تابعة لموقع ebay ثم دخل إلى موقع amazon الذي خلق كوكي جديدة برقم 1678 وأضافها إلى رسالة الجواب `Set-cookie: 1678`. عندما يعاود المستثمر الدخول من المتصفح نفسه إلى موقع amazon بعد أسبوع مثلاً فإن المتصفح (زبون HTTP) يرسل مع رسالة الطلب HTTP Request message رقم الكوكي المرتبطة بموقع amazon وهي `Cookie: 1678`. عندها يستطيع المخدم التعرف على المستثمر وعلى عملياته السابقة وإعادة المحتوى المناسب له.

3.2. المخدم الوكيل Proxy server وتخبئة الوب Web caching

يدعم بروتوكول HTTP المخدم الوكيل. المخدم الوكيل هو حاسوب يحفظ نسخة عن أجوبة الطلبات الحديثة. يرسل زبون HTTP طلب إلى المخدم الوكيل الذي يقوم باختبار الذاكرة الخابية لديه فإذا كان الجواب غير مخزن

ضمن الذاكرة الخابية فإنه يرسل الطلب إلى المخدم المطلوب. يجري أيضاً إرسال الأجوبة القادمة إلى المخدم الوكيل الذي يخزنها من أجل الطلبات المقبلة من زبائن أخرى.

يقلص المخدم الوكيل العبء عن المخدمات الأصلية ويقلل حركة المرور ويقلل زمن الاستجابة. يتطلب استخدام المخدم الوكيل إعداد الزبون للنفاذ إلى المخدم الوكيل بدلاً عن مخدّمات الإنترنت.

الجب الشرطي Conditional Get

تتمثل إحدى المشاكل التي تطرحها عملية التخبيئة في التأكد من كون النسخة الموجودة لدى مخدم التخبيئة ما تزال صالحة ولم تتغير بعد عند المخدم الأصلي. يوجد ضمن بروتوكول HTTP آلية تسمح لمخدم التخبيئة أو لأي زبون وب من التأكد من كون النسخة التي بحوزتهما ما تزال صالحة. تدعى هذه الآلية بالجب الشرطي Conditional Get. يدعى طلب HTTP بالاسم Conditional Get إذا (1) استخدم الطلب طريقة Get و (2) احتوت رسالة الطلب على سطر الترويسة If-Modified-Since.

لنأخذ المثال التالي: لنفرض أن المخدم الوكيل Proxy Server، الذي يعمل عادة كمخدم تخبيئة، أرسل الطلب التالي إلى مخدم الوب:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

ثم أرسل مخدم الوب رسالة الجواب التالية:

```
HTTP/1.1 200 OK
Date: Sat, 8 Oct 2011 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 7 Sep 2011 09:23:24
Content-Type: image/gif
(data data data data data ...)
```

يرسل مخدم التخبيئة الرسالة إلى المتصفح الطالب ويحتفظ بنسخة ضمن الخابية المحلية. تحتفظ الخابية أيضاً بالتاريخ الموجود ضمن سطر الترويسة Last-Modified مع الرسالة. إذا طلب متصفح ما الصفحة نفسها بعد عدة أيام وإذا كانت الصفحة ما تزال ضمن الخابية فإن مخدم التخبيئة يقوم بفحص صلاحية النسخة المحلية عن طريق إرسال طلب Conditional Get على الشكل التالي:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 7 Sep 2011 09:23:24
```

عندما يستلم مخدم الوب رسالة Conditional Get فإنه يتأكد فيما إذا كانت الصفحة قد تغيرت منذ التاريخ المطلوب وفي حالة الإيجاب يرسل الصفحة الحديثة أما إذا لم تتغير الصفحة فإن مخدم الوب يرسل المعلومات التالية:

```
HTTP/1.1 304 Not Modified
Date: Sat, 15 Oct 2011 15:39:29
Server: Apache/1.3.0 (Unix)
(empty entity body)
```

لاحظ أن مخدم الويب لا يرسل الصفحة المطلوبة إذا لم تتغير الأكر الذي من شأنه تقليل هدر الموارد بدون حاجة وتقليل زمن وصول الصفحة إلى الزبون وخاصةً إذا كان حجم الصفحة كبير .
تجدر الإشارة هنا إلى أن المخدم الأصلي يحدد مع كل جواب مدة صلاحية الجواب عن طريق الترويسة Expires أو Max-age ويستطيع مخدم التخزنة معرفة حداثة المعلومة المخزنة بالعودة إلى أحدهما علماً أن Max-age هو أقوى من Expires.

3. عملي

1.3 رسائل Get/response

سنحاول في هذا التمرين تحليل طريقتي Get و response باستخدام أداة wireshark كما يلي:

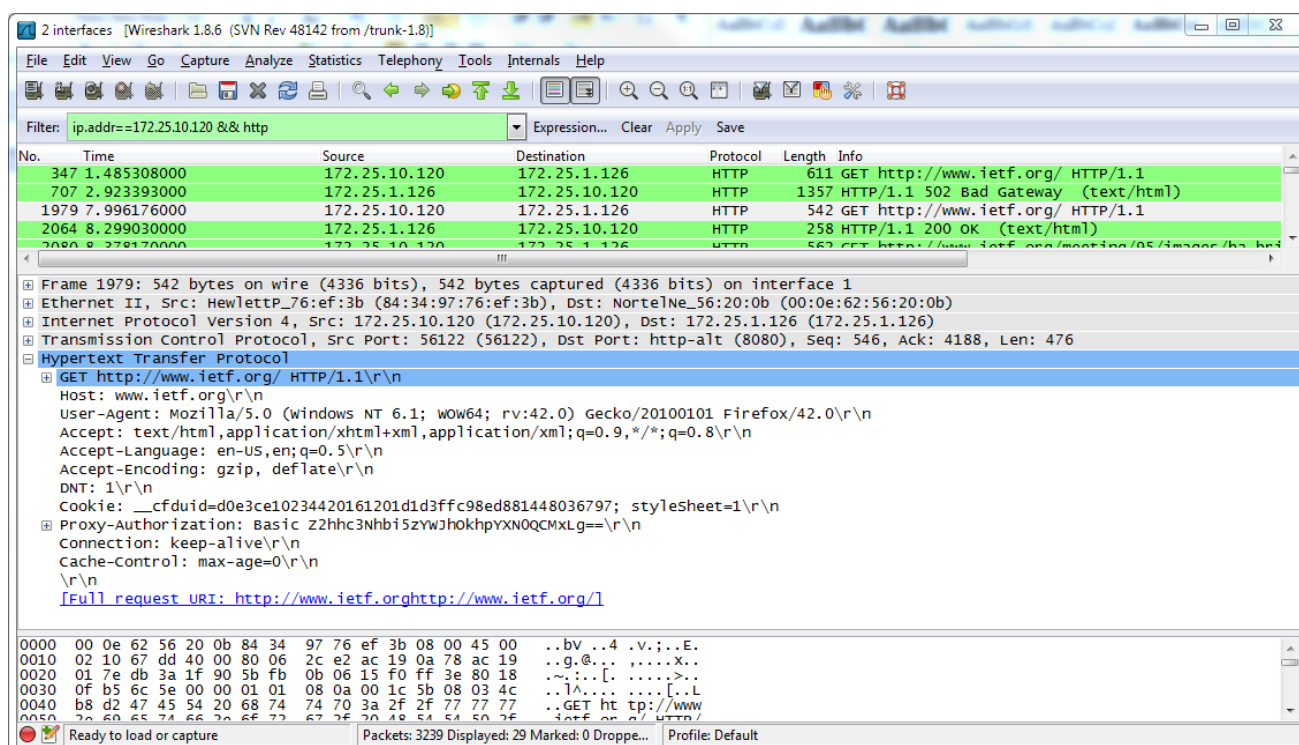
1. افتح متصفح الإنترنت

2. شغل wireshark وضمن Filter ضع http

3. افتح الصفحة www.ietf.org

4. أوقف تحليل الطرود ضمن wireshark

ستكون صفحة wireshark مشابهة للشكل التالي



أجب على الأسئلة التالية بعد النظر ضمن طرد السؤال Get من المتصفح إلى المخدم وطرد الجواب Response من المخدم إلى المتصفح:

1. هل يستخدم متصفحك النسخة HTTP 1.0 or 1.1؟ وما هي النسخة التي يستخدمها المخدم؟
2. ما هي اللغة التي يقبل استخدامها متصفحك؟
3. ما هو عنوان IP لحاسبك؟ وللمخدم؟
4. ما هي قيمة Status code التي أرجعها المخدم؟
5. متى جرى آخر تعديل على الصفحة المطلوبة؟
6. ما عدد البايتات التي أرجعها المخدم إلى المتصفح؟

2.3 استخدام HTTP conditional Get/response

حتى تتفد التمرين بالشكل الصحيح، يجب عليك أولاً حذف معلومات المتصفح الموجودة ضمن الذاكرة الخابية عن طريق تنفيذ ما يلي: Tools-> Clear Recent History ومن ثم اختيار Cache box بالنسبة للمتصفح Firefox و Delete File و Internet Options->Tools بالنسبة للمتصفح IE. يمكنك الآن البدء بالتمرين:

1. شغل المتصفح
2. شغل Wireshark
3. أدخل العنوان التالي ضمن URL: www.ietf.org
4. أضغط على الزر F5 لطلب "refresh"
5. أوقف Wireshark وصفي مدخلات Wireshark بحيث يكون البروتوكول هو HTTP

أجب عن الأسئلة التالية:

1. أبحث ضمن أول طلب Get مرسل من قبل متصفحك. هل ترى سطر "IF-modified-since" ضمن HTTP GET؟
2. افحص رسالة الجواب القادمة من المخدم. هل أرسل المخدم المحتوى المطلوب؟ كيف يمكنك التأكد من ذلك؟
3. أبحث الآن محتوى ثاني HTTP Get مرسل إلى المخدم نفسه. هل ترى السطر "if-modified-since" ضمن HTTP Get؟ في حالة الإجابة بنعم، ما هي المعلومات الإضافية التي تتبع هذه الترويسة؟
4. ما هي حالة الجواب Status Code المرسل كجواب للاستفسار الثاني؟ هل أرسل المخدم المحتوى المطلوب؟

3.3. عرض ملف كبير

سنحاول في هذا التمرين عرض ملف كبير ونسجل الملاحظات.

1. شغل المتصفح

2. شغل Wireshark

3. أدخل العنوان التالي: www.hiast.edu.sy

4. أوقف Wireshark

أجب على الأسئلة التالية:

1. ما عدد الاستفسارات من نوع Get request التي أرسلها المتصفح؟

2. ما ترتيب الاستفسار Get الذي يطلب الغرض

GET /sites/all/modules/languageicons/flags/ar.png

3. ما هو رقم الطرد الذي يحمل رسالة الجواب Status code على أول رسالة GET

4. ما هي قيمة الجواب والجملة Status code and phrase

5. ما هو عدد الطرود الكلية بما فيها طرود TCP اللازمة لإرسال الطلبات واستلام الأجوبة؟

6. ما هي أرقام البوابات المستخدمة TCP source and destination ports ضمن رسائل

طلبات Get ورسائل الجواب عليها؟

4.3. استيقان HTTP

لنحاول زيارة موقع وب يطلب اسم مستخدم وكلمة مرور ونفحص تسلسل رسائل HTTP المتبادلة.

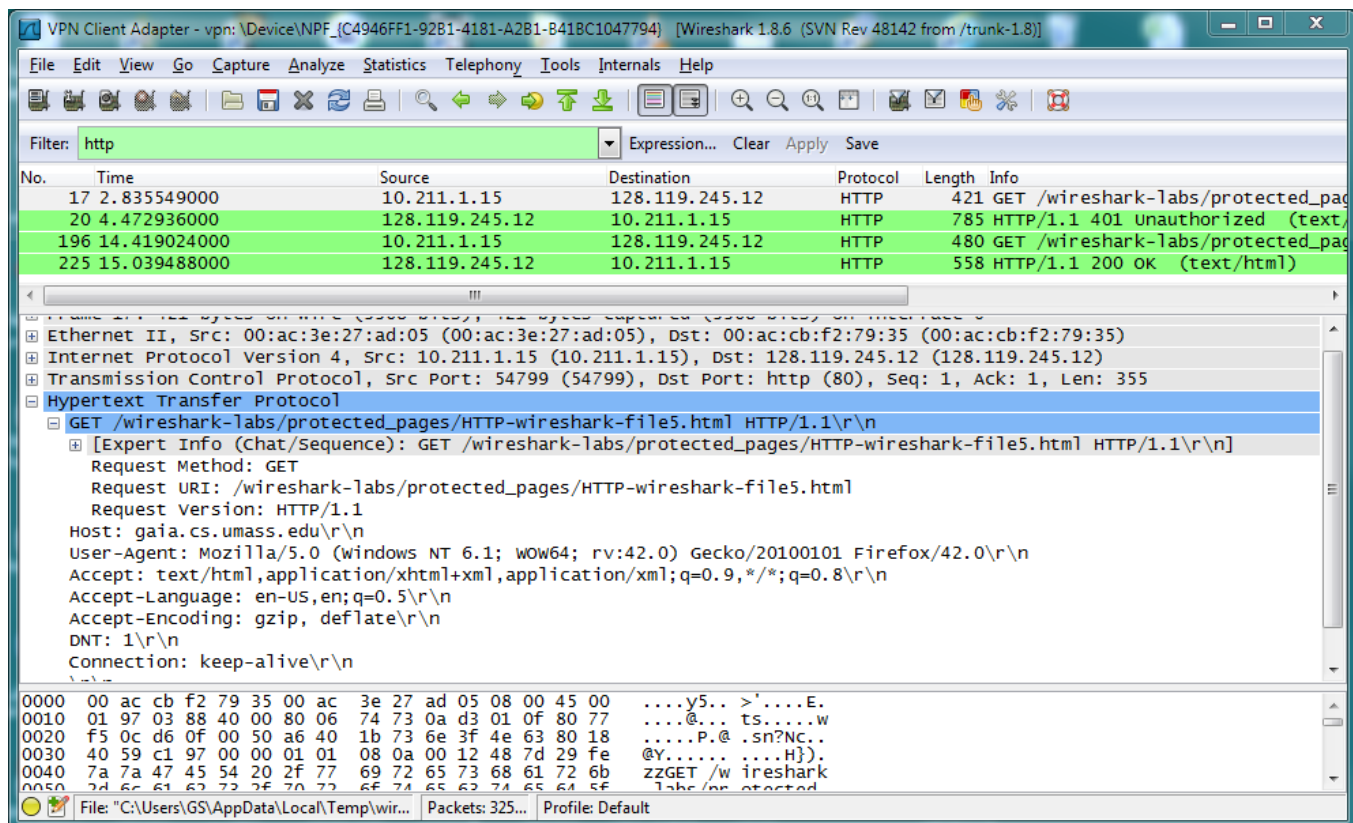
طبعاً سوف نشغل Wireshark (بعد حذف history مع كل محاولة جديدة) ثم ندخل العنوان التالي

ضمن المتصفح

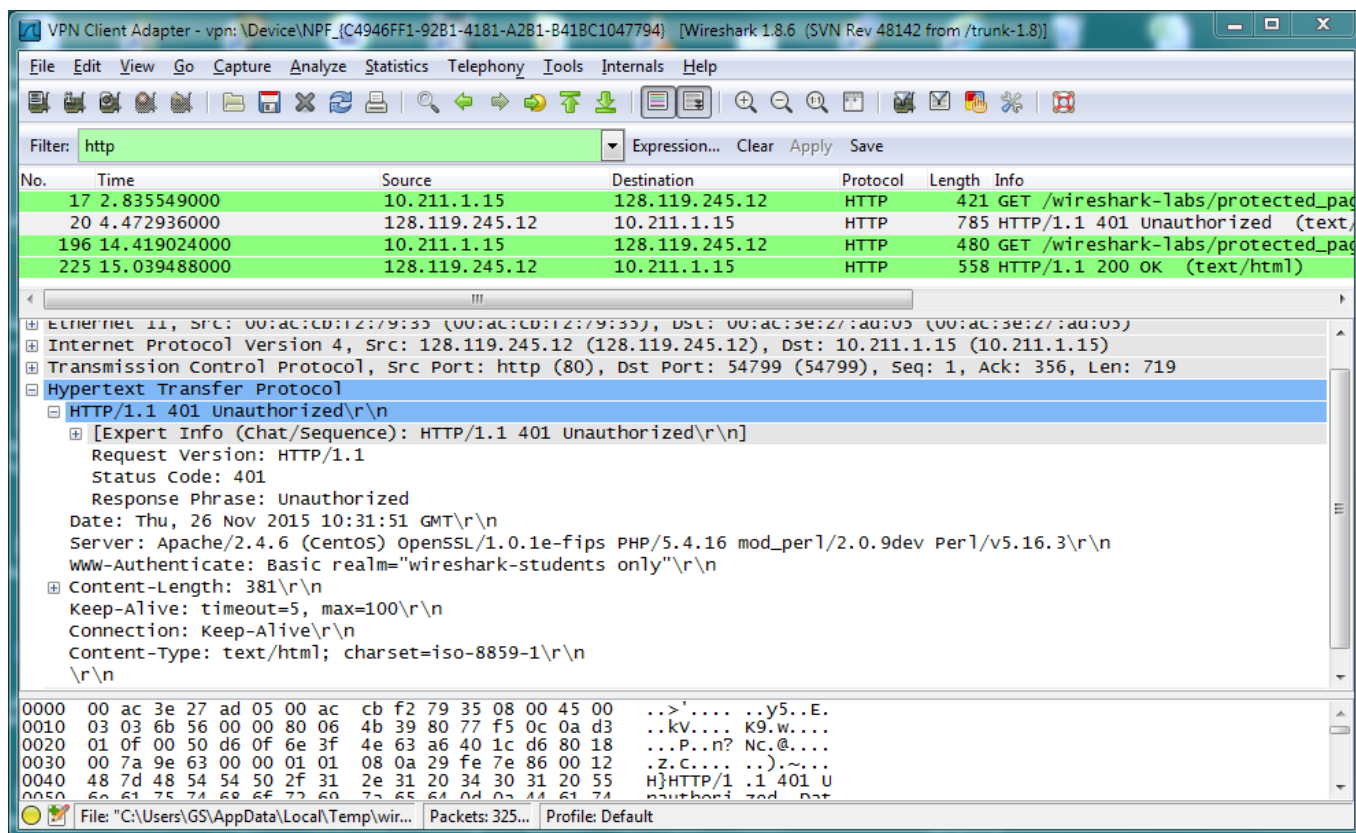
http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html

أدخل اسم المستخدم "wireshark-students" وكلمة المرور "network" وافحص نتائج محلل

الطرود Wireshark. تبين الأشكال التالية تسلسل إرسال الرسائل:

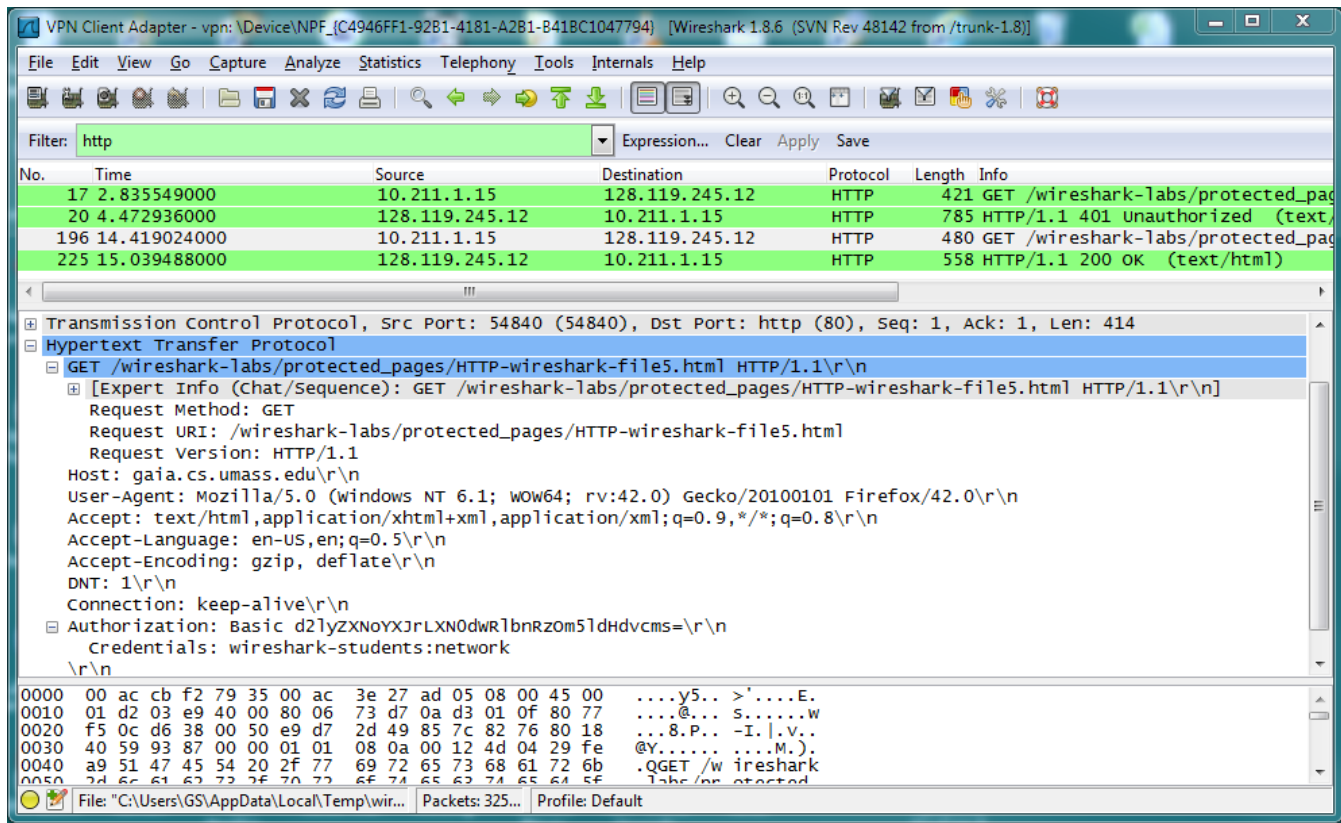


الشكل 15- الطرد الأول



الشكل 16- طرد الجواب

نلاحظ أن طرد الجواب يحمل الرقم 401 Unauthorized



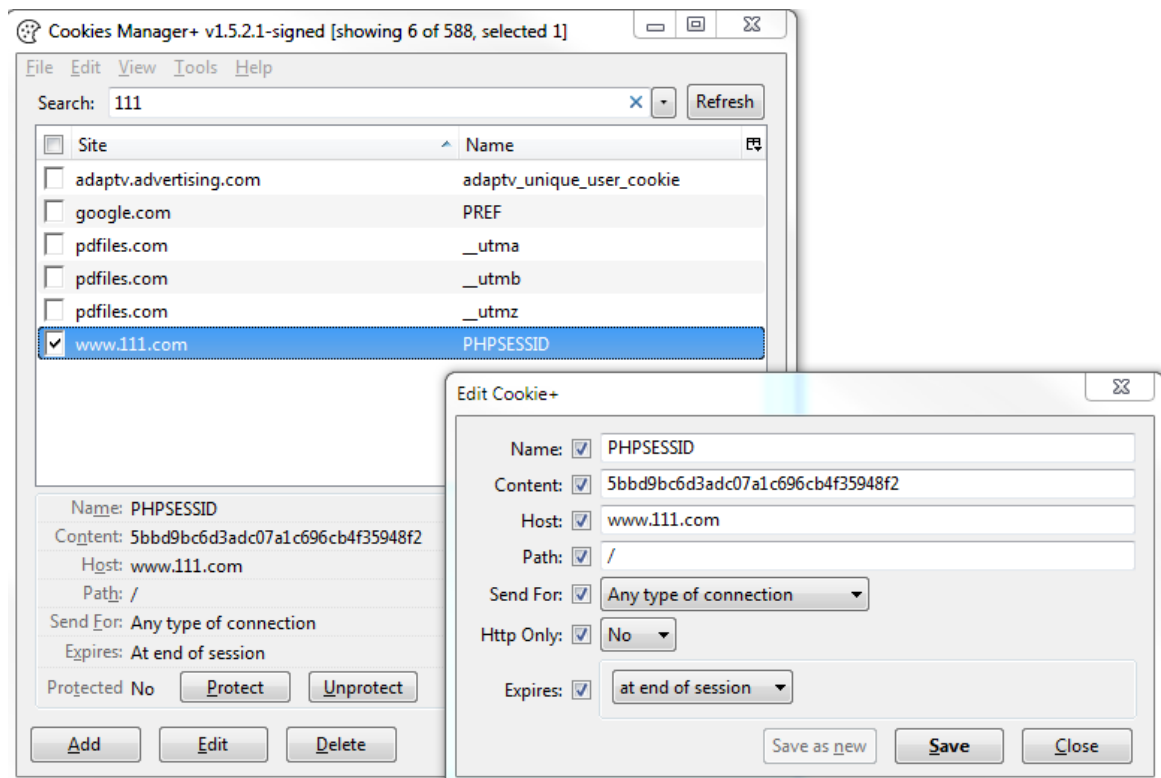
الشكل 17- طرد HTTP يحمل معلومات الاستيقان

الطرد المبين في الشكل السابق يحمل اسم المستخدم وكلمة المرور المطلوبة للموقع. لاحظ أن معلومات الاستيقان d2lyZXN... عندما تكون من النوع Basic تكون مرمزة وفق الترميز Base64 وليس مشفرة حيث يعرض السطر التالي نتيجة فك الترميز. بفضل هنا استخدام بروتوكول Digest الذي لا يرسل كلمة المرور عيّن الطرفين ويكون أكثر أماناً.

الطرد الأخير يظهر نجاح عملية الاستيقان والسماح بالدخول وعرض المعلومات المطلوبة. يطلب هنا من الطالب إعادة الخطوات السابقة والتأكد من الطرود الأربعة التي يجري إرسالها ومعلومات الاستيقان.

5.3. إدارة الكوكيز

ينصح هنا بتحميل Firefox Cookies Manager+ add on. ندخل إلى الموقع www.111.com ثم نستدعي Cookies Manager+ فتظهر لنا صفحة مثل الشكل التالي:



يبين الشكل السابق الكوكيز المضافة ومعلومات عنها. على الطالب هنا أن يشغل Wireshark ومن ثم الدخول إلى الموقع www.111.com/dns وفحص تسلسل الطرود وكتابة المعلومات الخاصة بالكوكيز عند أول دخول والجواب من المخدم وعند ثاني دخول إلى المخدم نفسه.